```
from torchvision.datasets import EMNIST

train_dataset = EMNIST(
    root='./data',
    split='balanced',
    train=True,
    download=True
)

test_dataset = EMNIST(
    root='./data',
    split='balanced',
    train=False,
    download=True
)

print("Dataset Downloaded Successfully")
```

```
100%|██████████| 562M/562M [00:02<00:00, 205MB/s]
Dataset Downloaded Successfully
```

**Step 1 — Libraries Import**

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

**Step 2 — Dataset Load**

```
transform = transforms.Compose([
    transforms.ToTensor()
])

train_dataset = datasets.EMNIST(
    root='./data',
    split='balanced',
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.EMNIST(
    root='./data',
    split='balanced',
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

print("Dataset Loaded Successfully")
```

```
Dataset Loaded Successfully
```

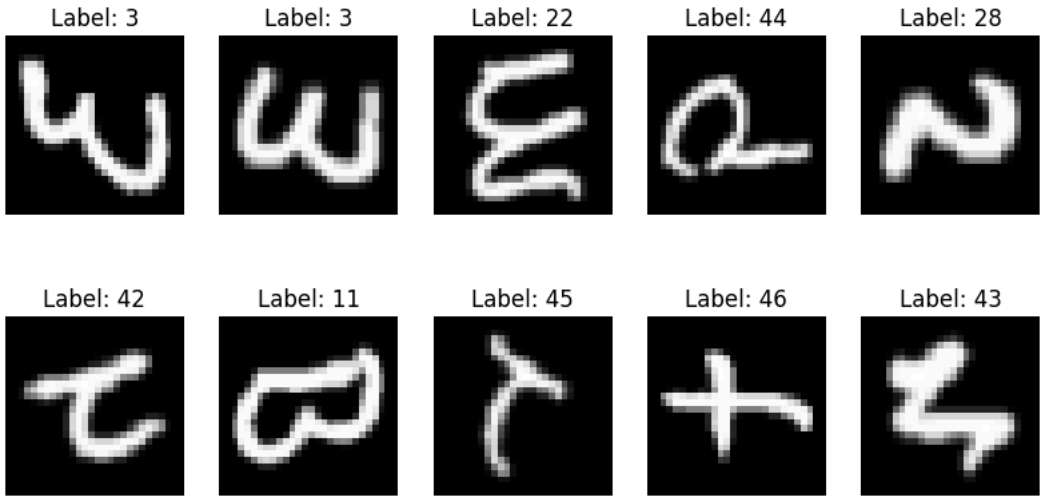
**Step 3 — Sample Images Show**

```
images, labels = next(iter(train_loader))

plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(images[i].squeeze(), cmap='gray')
    plt.title(f"Label: {labels[i].item()}")
    plt.axis('off')
```

```
plt.show()
```




**Step 4 — CNN Model**

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)

        self.pool = nn.MaxPool2d(2,2)

        self.fc1 = nn.Linear(64*5*5, 128)
        self.fc2 = nn.Linear(128, 47)

        self.relu = nn.ReLU()

    def forward(self, x):

        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))

        x = x.view(-1, 64*5*5)

        x = self.relu(self.fc1(x))
        x = self.fc2(x)

        return x

model = CNN()

print(model)
```

```
CNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=1600, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=47, bias=True)
  (relu): ReLU()
)
```

**Step 5 — Loss & Optimizer**

```
criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=0.001)
```

**Step 6 — Training**

```
epochs = 5

for epoch in range(epochs):

    model.train()

    total_loss = 0

    for images, labels in train_loader:

        outputs = model(images)

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)

    print("Epoch:", epoch + 1, "Loss:", avg_loss)
```

```
Epoch: 1 Loss: 0.3166306812022467
Epoch: 2 Loss: 0.2900010526763466
Epoch: 3 Loss: 0.26910329471493877
Epoch: 4 Loss: 0.2488789673529088
Epoch: 5 Loss: 0.23047989405273242
```

**Step 7 — Accuracy Check**

```
correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        outputs = model(images)

        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

print(f"Test Accuracy: {accuracy:.2f}%")
```

```
Test Accuracy: 87.27%
```

**Step 8 — Save Model**

```
torch.save(model.state_dict(), "handwritten_model.pth")

print("Model Saved Successfully")
```

```
Model Saved Successfully
```

**Final Prediction Testing**

```
images, labels = next(iter(test_loader))

outputs = model(images)

_, predicted = torch.max(outputs, 1)

plt.figure(figsize=(12,6))

for i in range(10):
```

```
        plt.subplot(2,5,i+1)
        plt.imshow(images[i].squeeze(), cmap='gray')
        plt.title(f"P: {predicted[i].item()} | T: {labels[i].item()}")
        plt.axis('off')
    plt.show()
```